杰理 AD1xN-45678_MIDI
应用说明文档

珠海市杰理科技股份有限公司
Zhuhai Jieli Technologyco.,LTD
版权所有，未经许可，禁止外传

2023 年 8 月 16 日

修改日志

版本	日期	描述
1.03	2023 / 8 / 16	添加 midi 解码音符结束回调所有音符说明
1.02	2023 / 7 / 11	添加限幅器说明
1.01	2022 / 11 / 8	改动与 AD14/AD15/AC104 V1.4.0 SDK 同步
更新： ● 新增 midi 外部音量功能说明； ● 新增 midi 琴 note_off 函数衰减参数说明； ● 新增 midi 琴获取指定通道正在播放的 key 函数 void get_midi_ctrl_play_key(music_play_obj *hdl, u8 chn)说明；		
1.00	2022 / 7 / 6	完成第一版 midi 说明文档
更新： ● 建立初始版本 ● 定义文档格式		

ZHUHAI JIELI TECHNOLOGY CO.,LTD

目录

第一章 MIDI 应用 6

1.1 MIDI 功能介绍 6
1.1.1 MIDI 简介 6
1.1.2 SDK 中 midi 模式的文件依赖 6
1.2 MIDI 乐谱解码 7
1.2.1 MIDI 乐谱解码初始化与 CMD 配置介绍 7
1.2.1.1 MIDI 乐谱解码运行步骤 7
1.2.1.2 初始化的配置 8
1.2.1.3 MIDI 乐谱解码的 CMD 配置 9
1.2.1.4 MIDI 在运行中动态配置功能示例 10
1.2.2 解码方式（CMD_MIDI_CTRL_MODE） 11
1.2.3 ONE KEY ONE NOTE 模式 12
1.2.3.1 打开 OKON 模式 12
1.2.3.2 OKON 的工作模式（CMD_MIDI_OKON_MODE） 12
1.2.3.3 设置主旋律 14
1.2.3.4 快速打断功能 14
1.2.3.5 中途切换 OKON 模式 14
1.2.4 主旋律乐器的选择（CMD_MIDI_SET_CHN_PROG） 15
1.2.5 乐谱中添加标记(MARK)功能 16
1.2.5.1 MARK 回调函数 16
1.2.5.2 两个 MARK 之间循环播放 16
1.2.6 其他回调功能 17
1.2.6.1 主旋律的音符回调功能 17
1.2.6.2 主旋律的音符结束回调功能 17
1.2.6.3 节拍(BEAT)的回调 18
1.2.6.4 小节的回调 18
1.2.6.5 清狗的回调 18
1.2.7 外部音量功能（CMD_MIDI_SET_EX_VOL） 18
1.2.8 移半音功能（CMD_MIDI_SET_SEMITONE） 19
1.2.9 更改节奏和衰减（CMD_MIDI_CTRL_TEMPO） 19
1.2.10 小节回退功能（CMD_MIDI_SEEK_BACK_N） 20
1.2.11 MIDI 解码函数接口 21
1.2.11.1 函数 void midi_decode_init(void) 21
1.2.11.2 函数 int music_midi_ctrl_config(music_play_obj *hdl, u32 cmd, void *parm) 21
1.2.11.3 函数 void midi_okon_mode_set(music_play_obj *music_obj, u8 mode) 22
1.2.11.4 函数 void midi_okon_goon(music_play_obj *music_obj) 22
1.2.11.5 函数 void midi_init_info(MIDI_INIT_STRUCT *init_info) 22
1.3 MIDI 琴 23
1.3.1 MIDI 琴初始化 23
1.3.1.1 初始化过程 23
1.3.1.2 注意要点 23
1.3.2 MIDI 琴音符回调功能 24

ZHUHAI JIELI TECHNOLOGY CO.,LTD

1.3.3 MIDI 琴函数接口 *24*
1.3.3.1 函数 void midi_ctrl_decode_init(void) 24
1.3.3.2 函数 int music_midi_ctrl_set_prog(music_play_obj *hdl, u8 prog, u8 chn) 24
1.3.3.3 函数 int music_midi_ctrl_pitch_bend(music_play_obj *hdl, u16 pitch_val, u8 chn) 24
1.3.3.4 函数 int music_midi_ctrl_config(music_play_obj *hdl, u32 cmd, void *parm) 25
1.3.3.5 函数 int music_midi_ctrl_note_on(music_play_obj *hdl, u8 nkey, u8 nvel, u8 chn) 25
1.3.3.6 函数 int music_midi_ctrl_note_off(music_play_obj *hdl, u8 nkey, u8 chn, u16 decay_time) 26
1.3.3.7 函数 int music_midi_ctrl_vel_vibrate(music_play_obj *hdl, u8 nkey, u8 vel_step, u8 vel_rate, u8 chn) 26
1.3.3.8 函数 int midi_ctrl_play_end_deal(music_play_obj *hdl) 26
1.3.3.9 函数 void midi_ctrl_callback_init(music_play_obj *hdl, u32(*melody_callback)(void *, u8, u8), u32(*melody_stop_callback)(void *, u8)) 27
1.3.3.10 函数 void get_midi_ctrl_play_key(music_play_obj *hdl, u8 chn) 27
1.4 midi 限幅器 28
1.4.1. 使用说明 *28*
1.4.2. 参数说明 *28*
第二章 MIDI 工具 30
2.1 MIDI 工具主界面 30
2.1.1 MIDI 版本说明 *30*
2.1.2 MIDI 工具简介 *30*
2.1.3 主界面-乐器种类 *31*
2.1.4 主界面-选中乐器 *31*
2.2 MIDI 转换 32
2.2.1 选择需要转换的 MIDI 文件 *32*
2.2.2 输出目录 *33*
2.2.3 转换 *33*
2.2.4 导出库 *34*
2.3 MIDI 音色库压缩 35
2.3.1 压缩流程 *35*
2.3.2 压缩信息查询 *38*
2.3.3 还原压缩配置内容 *40*
2.4 MIDI 导入 SF2 音色库 40
2.4.1 添加或删除 SF2 文件 *40*
2.4.2 SF2 音色库信息查询 *41*
2.4.3 还原 SF2 音色库内容 *42*
2.5 MIDI WAV 编辑 43
2.5.1 WAV 编辑界面简介 *43*
2.5.2 波形编辑 *44*
2.5.3 参数配置 *45*
2.5.4 包络编辑 *47*
2.5.5 波形保存 *47*
2.5.6 WAV 编辑信息查询 *48*
2.5.7 还原 WAV 编辑内容 *48*
2.6 按键与乐器表 49

ZHUHAI JIELI TECHNOLOGY CO.,LTD

2.6.1 按键表.....49
2.6.2 乐器表.....49

ZHUHAI JIELI TECHNOLOGY CO.,LTD

第一章 MIDI 应用

1.1 MIDI 功能介绍

1.1.1 MIDI 简介

MIDI 全称是乐器数字接口(Musical Instrument Digital Interface)，是 20 世纪 80 年代初为解决电声乐器之间的通信问题而提出的，编曲界最广泛的音乐标准格式。

MIDI 用音符的数字控制信号来记录音乐。一首完整的 MIDI 音乐只有几十 KB 大，而能包含数十条音乐轨道。其传输的不是声音信号，而是音符、控制参数等指令，它指示 MIDI 设备要做什么，怎么做，如演奏哪个音符、多大音量等。

SH54 / 55 支持 MIDI 乐谱解码以及 MIDI 琴两种应用。其中：

MIDI 乐谱解码有 4 种方式：

① **正常解码模式**

② **OKON（ONE KEY ONE NOTE）模式**

③ **只推消息不出声模式**

④ **外部音源模式**

MIDI 琴有 1 种方式：

① **琴键解码模式**

1.1.2 SDK 中 midi 模式的文件依赖

SDK 中 midi 乐谱解码模式需要 midi 乐谱，以及 midi 音色库，其中 midi 乐谱通过打包工具制作成 dir_midi 文件，midi 音色库通过打包工具制作成 midi_cfg 文件。

SDK 中 midi 琴模式需要 midi 音色库，midi 音色库通过打包工具制作成 midi_ctrl_prog 文件。

以上打包文件名可以自定义，在 SDK 中 midi 初始化代码中修改相应名称至可识别即可。midi 音色库文件可共用，详情想看第二章 midi 工具使用介绍。

ZHUHAI JIELI TECHNOLOGY CO,LTD

1.2 MIDI 乐谱解码

MIDI 乐谱解码的过程为，初始化后，打开 MIDI 乐谱文件获取到音符，再访问音色库找到对应的乐器音色进行解码，不断获取音符并重复该过程直到乐谱结束。本节讲介绍 MIDI 解码的初始化、MIDI 解码的功能配置以及注意点。

1.2.1 MIDI 乐谱解码初始化与 CMD 配置介绍

1.2.1.1 MIDI 乐谱解码运行步骤

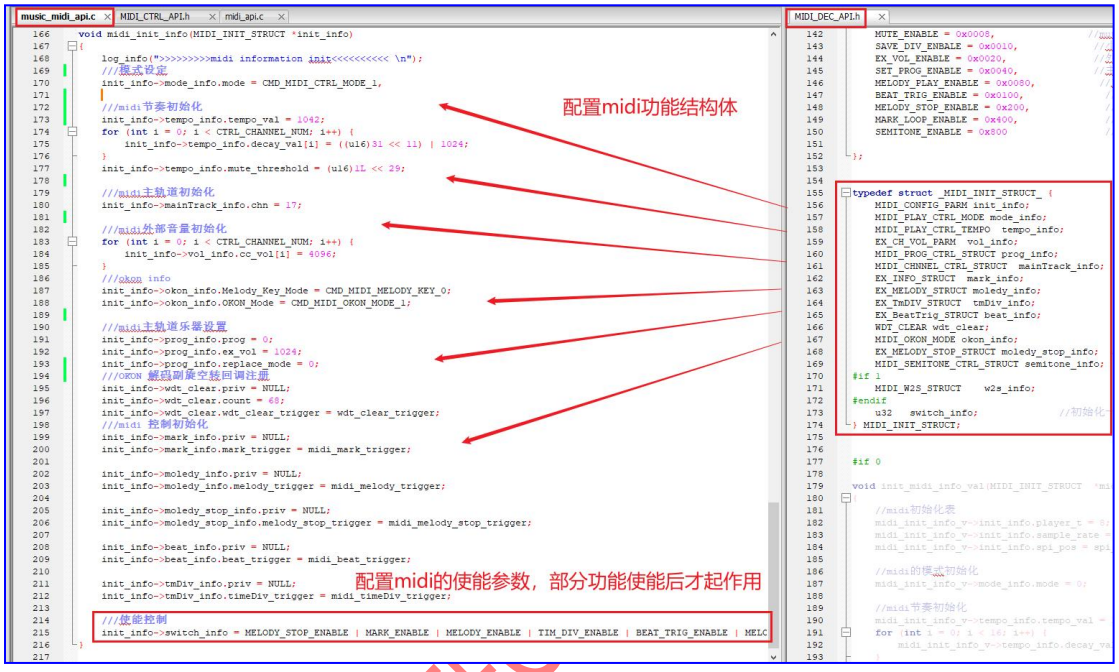
1) 在系统初始化时会调用 midi_decode_init(void)函数记录 MIDI 音色库文件的信息；midi 乐谱解码 MAX_DEC_PLAYER_CNT 同时发声的 key 数默认为 8。
2) 调用 decoder_init(void)初始化解码器；
3) 打开 midi 乐谱文件并调用函数 decoder_io(void *pfile, u32 dec_ctl, dp_buff *dbuff, u8 loop)进行解码，该函数 dec_ctl 参数传入 BIT_MIDI；
4) midi 解码初始化中会调用函数 void midi_decode_api(void *p_file, void **ppdec, void *p_dp_buf)，初始化 MIDI 解码信息，比其他解码格式多了配置参数这一步！
5) 配置参数会调用函数 midi_init_info(MIDI_INIT_STRUCT *init_info)，进行 MIDI 功能的初始化，包括主旋律选择、外部音量设置、所有回调函数的注册等，最后使能对应的控制使能位。
6) 通过 CMD_INIT_CONFIG 命令将 midi 初始化的信息传入解码器中，到此 midi 功能初始化完成。
7) 解码器启动，dac 开始发声；



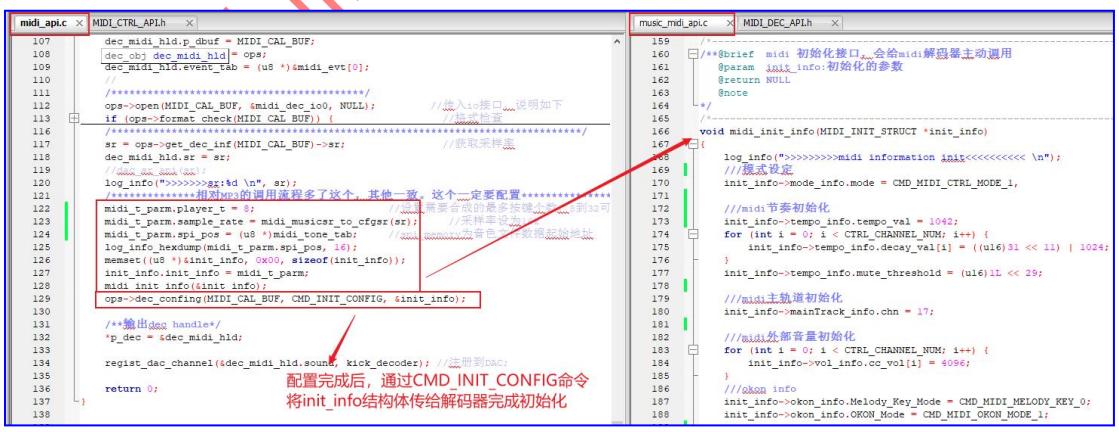
ZHUHAI JIELI TECHNOLOGY CO.,LTD

1.2.1.2 初始化的配置

MIDI 解码功能的初始化在函数 void midi_init_info(MIDI_INIT_STRUCT *init_info)中进行。该函数中会配置 MIDI_INIT_STRUCT 结构体内包含的成员。配置功能结构体后，还需要配置功能对应的控制使能参数，sdk 中默认的 MIDI 功能配置如图所示：



配置完成后，通过 CMD_INIT_CONFIG 命令将配置完成的 MIDI_INIT_STRUCT 结构体传给解码器，如下图所示：



1.2.1.3 MIDI 乐谱解码的 CMD 配置

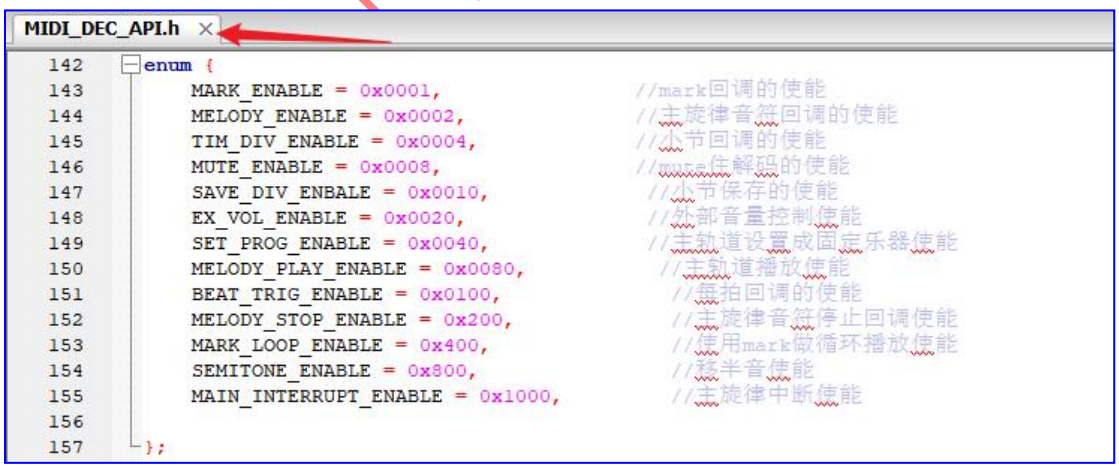
1) CMD 命令配置说明

MIDI 乐谱解码可以通过 CMD 实时配置功能。首先配置 CMD 对应的功能结构体，再调用函数 int music_midi_ctrl_config(music_play_obj *hdl, u32 cmd, void *parm)，函数参数分别传入 midi 解码句柄、CMD 命令以及配置好的功能结构体指针即可实现，MIDI 的 CMD 命令及对应的功能结构体如下图所示。



2) 控制使能参数说明（CMD_MIDI_SET_SWITCH 命令）

部分功能配置结束后，还需要置上相应的使能位才能生效（初始化中已经置上的使能位不需要再次使能）。使能参数是一个 u32 变量，每一 bit 代表一个功能，该 bit 置 1/置 0 会打开/关闭对应的功能。



MIDI 功能使能的方式有两种，一种是在 MIDI 初始化中配置 MIDI_INIT_STRUCT 结构体的 switch_info 成员，并调用 CMD_INIT_CONFIG 命令配置初始化后生效。

另一种时通过调用上一节函数 music_midi_ctrl_config()，分别传入 MIDI 解码句柄、CMD_MIDI_SET_SWITCH 命令以及配置好的使能参数。注意，该命令传入的使能参数是所

有功能的使能位，单独操作某一使能位时不能影响到其他 bit，否则会影响到其他功能。可以沿用 MIDI 初始化的全局变量 init_info->switch_info 来管理使能位。



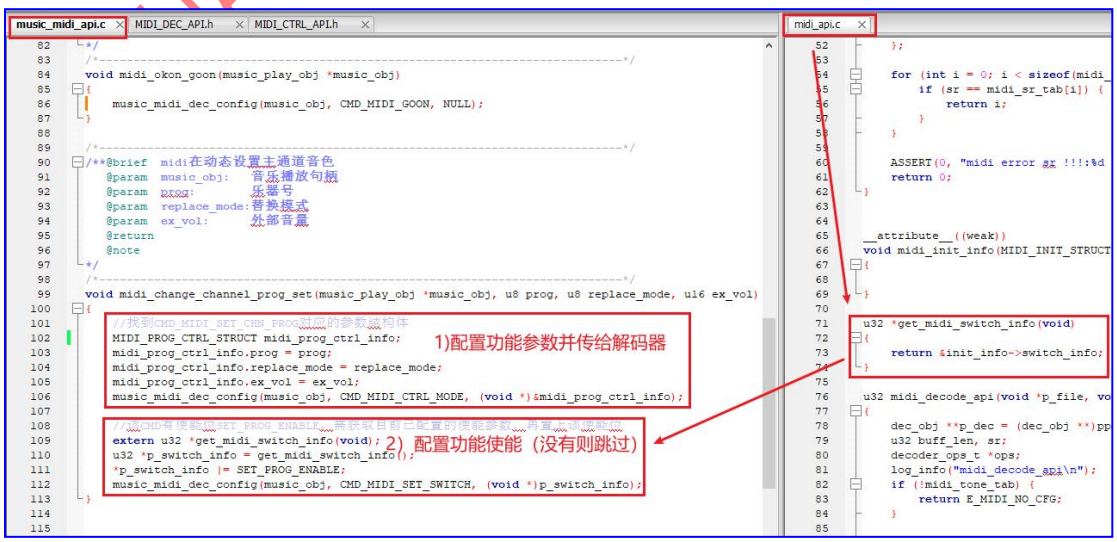
1.2.1.4 MIDI 在运行中动态配置功能示例

本小节以在 MIDI 运行中配置主旋律乐器为例，其他功能配置可参照该例程；

1) 首先查看 MIDI 支持的 CMD 命令 CMD_MIDI_SET_CHN_PROG，并找到它对应的结构体参数 MIDI_PROG_CTRL_STRUCT，以及对应的使能位 SET_PROG_ENABLE（无使能位则忽略）。

2) 根据需要配置结构体，并通过 music_midi_dec_config()函数将 CMD 和配置好的结构体指针传给解码器；

3) 若该功能有使能位且 MIDI 初始化中未使能，则需要使能对应的功能。首先获取全局变量 init_info->switch_info 的值得到所有使能配置，再置上对应的使能位，通过 CMD_MIDI_SET_SWITCH 命令将使能参数传给解码器，该功能配置就会生效。



ZHUHAI JIELI TECHNOLOGY CO.,LTD

1.2.2 解码方式（CMD_MIDI_CTRL_MODE）

MIDI 解码方式共有 4 种，分别是：

① **正常解码模式**

该模式即正常解码 MIDI 乐谱文件。

② **OKON（ONE KEY ONE NOTE）模式**

该模式即用户通过按键驱动 MIDI 文件解码，通过调用函数实现按一次按键播放一段音符。

③ **只推消息不出声模式**

该模式暂不支持。

④ **外部音源替换模式**

该消息主要应用为使用录音音源替换乐谱，录音文件需要经过处理。

其各自对应参数如下：

```
MIDI_DEC_API.h
58
59  enum {
60      CMD_MIDI_CTRL_MODE_0    = 0X00,     //正常解码
61      CMD_MIDI_CTRL_MODE_1    = 0X01,     //OKON 模式
62      CMD_MIDI_CTRL_MODE_2,               //只推消息，不出声
63      CMD_MIDI_CTRL_MODE_W2S              //外部音源
64  };
65
```

MIDI 初始化中配置 init_info->mode_info 即可选择默认的解码方式，如下图所示，sdk 默认选择了 OKON 模式。

```
MIDI_DEC_API.h | midi_ctrl_api.c | music_midi_api.c
159  /*-----------------------------------------------------------------
160  /**@brief  midi 初始化接口，会给midi解码器主动调用
161      @param  init_info:初始化的参数
162      @return NULL
163      @note
164  */
165  /*-----------------------------------------------------------------
166  void midi_init_info(MIDI_INIT_STRUCT *init_info)
167  {
168      log_info(">>>>>>>>>midi information init<<<<<<<<< \n");
169      ///模式设定
170      init_info->mode_info.mode = CMD_MIDI_CTRL_MODE_1,
171
```

初始化配置之后调用 midi_okon_mode_set(music_play_obj *music_obj, u8 mode)函数，可以修改 MIDI 的解码方式；详情请看《3.3.3.函数 void midi_okon_mode_set(music_play_obj *music_obj, u8 mode)》说明。



1.2.3 ONE KEY ONE NOTE 模式

1.2.3.1 打开 OKON 模式

调用 midi_okon_mode_set()，传入 CMD_MIDI_CTRL_MODE_1 即可切换到 OKON 模式。使用 OKON 模式需要置上 MELODY_PLAY_ENABLE 使能位。

1.2.3.2 OKON 的工作模式（CMD_MIDI_OKON_MODE）

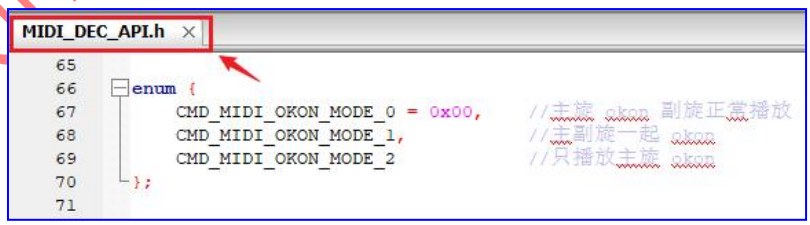
OKON 的工作模式共有三种，分别是：

① **主旋律 OKON 副旋律正常播放；**

② **主副旋律一起 OKON；**

③ **只有主旋律 OKON 副旋律不发声；**

其各自对应的参数如下：



其中，只有主旋律 OKON 副旋律不发声的工作模式，在解码过程中有可能长时间占用 CPU，所以需要在 wdt_clear_trigger()函数中调用清狗函数，否则有可能会导致系统复位！

ZHUHAI JIELI TECHNOLOGY CO.,LTD

```
127  u32 wdt_clear_trigger(void *priv)
128  {
129      wdt_clear();          回调清狗函数
130  //      log_char('w');
131      /* log_info("wdt_clear_trigger \n"); */
132      return 64;
133  }
134
```

OKON 按键按下后可以回调的音符，该功能需要置上 MELODY_ENABLE 使能位。回调的方式共有两种，分别是：

① **回调当前音符**

② **回调下一个音符位**

其各自对应参数如下：

MIDI_DEC_API.h

```
71
72  enum {
73      CMD_MIDI_MELODY_KEY_0 = 0x00,    //melody_trigger 回调当前音符
74      CMD_MIDI_MELODY_KEY_1            //melody_trigger 回调下一个音符
75  };
76
```

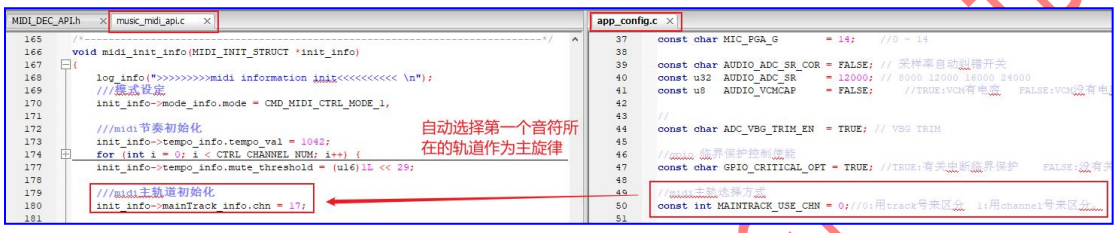
MIDI 初始化中配置 init_info->okon_info 即可选择默认的 OKON 工作模式，如下图所示，sdk 默认选择了主旋律 OKON 副旋律正常播放和回调当前音符。

MIDI_DEC_API.h | music_midi_api.c

```
165  /*----------------------------------------------------------------------
166  void midi_init_info(MIDI_INIT_STRUCT *init_info)
167  {
168      log_info(">>>>>>>>>midi information init<<<<<<<<<< \n");
169      ///模式设定
170      init_info->mode_info.mode = CMD_MIDI_CTRL_MODE_1,
171
172      ///midi节奏初始化
173      init_info->tempo_info.tempo_val = 1042;
174      for (int i = 0; i < CTRL_CHANNEL_NUM; i++) {
177      init_info->tempo_info.mute_threshold = (u16)1L << 29;
178
179      ///midi主轨道初始化
180      init_info->mainTrack_info.chn = 17;
181
182      ///midi外部音量初始化
183      for (int i = 0; i < CTRL_CHANNEL_NUM; i++) {
186
187      ///okon info
188      init_info->okon_info.Melody_Key_Mode = CMD_MIDI_MELODY_KEY_0;
189      init_info->okon_info.OKON_Mode = CMD_MIDI_OKON_MODE_0;
190
```

1.2.3.3 设置主旋律

OKON 模式需要确定主旋律来播放音符。初始化时，系统会判断 const 变量 MAINTRACK_USE_CHN 的值，值为 0 通过轨道(track)号选择主旋律，值为 1 通过通道(channal)号选择主旋律。修改 init_info->mainTrack_info.chn 的值可以在初始化中确定主旋律，有效范围是 0~15。若让系统自动选择第一个音符所在轨道或通道为主旋律，则选择 17。Sdk 中的默认配置如下：



注意，通过轨道(track)号选择主旋律时，mainTrack_info.chn 的值与乐谱制作软件的轨道号一致；通过通道(channal)号选择主旋律时，mainTrack_info.chn 的值需要比乐谱制作软件的通道号小 1，即 mainTrack_info.chn 的值为 0 时，对应乐谱中的 1 号通道作为主旋律；

MIDI 乐谱制作时，一个轨道只能对应一个通道，但是一个通道可以对应多个轨道；例如音轨 1、音轨 2 对应通道 1，音轨 3 对应通道 2，那么以轨道(track)号选择主旋律时，mainTrack_info.chn 设置为 1，则音轨 1 为主旋律；以通道(channal)号选择主旋律时，mainTrack_info.chn 设置为 0，则通道 1 对应的音轨 1 和音轨 2 一起成为主旋律。

1.2.3.4 快速打断功能

在只有主旋律 OKON 副旋律不发声的模式下，可以置上 MAIN_INTERRUPT_ENABLE 使能位开启快速打断功能。上一个音符未结束而下一个音符到来时，未开启快速打断，两个音符会叠加在一起直至上一个音符结束；开启快速打断后，会加快上一个音符的衰减，使其迅速消失减少叠加时间。

1.2.3.5 中途切换 OKON 模式

中途切换 MIDI 模式可以调用函数 midi_okon_mode_set()，并将 mode 设置为 OKON 模式(CMD_MIDI_CTRL_MODE_1)。OKON 模式下需要调用 midi_okon_goon()函数启动下一个音符。sdk 中应用示如图所示：

ZHUHAI JIELI TECHNOLOGY LTD



1.2.4 主旋律乐器的选择（CMD_MIDI_SET_CHN_PROG）

MIDI 可以强制指定主旋律的乐器，可以做到实时变更主旋律的乐器以及相关音量配置，使用该功能需要置上 SET_PROG_ENABLE 使能位，没有使能则使用 MIDI 乐谱上默认的乐器。其功能结构体 MIDI_PROG_CTRL_STRUCT 包括：

① **prog**

乐器号，与我司 midi 制作工具中的乐器号对应；

② **replace_mode**

替换模式，选 0 是只更变主旋律的乐器，选 1 是更变所有旋律的乐器。暂不支持单独更变副旋律的乐器；

③ **ex_vol**

音量配置，该值会将音量改变为原来的(ex_vol / 1024)倍；值为 1024 则不改变音量；

Sdk 中的默认配置如下：



该功能可以在 MIDI 运行中动态设置，调用函数 music_midi_ctrl_config()，并将 CMD_MIDI_SET_CHN_PROG 命令和配置好的结构体参数传入；置上 SET_PROG_ENABLE 使能位即可。详情请看《2.1.3MIDI 乐谱解码的 CMD 配置》说明。

JIELI TECHNOLOGY CO.,LTD

1.2.5 乐谱中添加标记(MARK)功能

MIDI 解码能够识别到 MIDI 乐谱文件上标准的 MARK 标记。遇到 MARK 标记时支持回调函数，而且支持两个 MARK 之间进行循环播放。

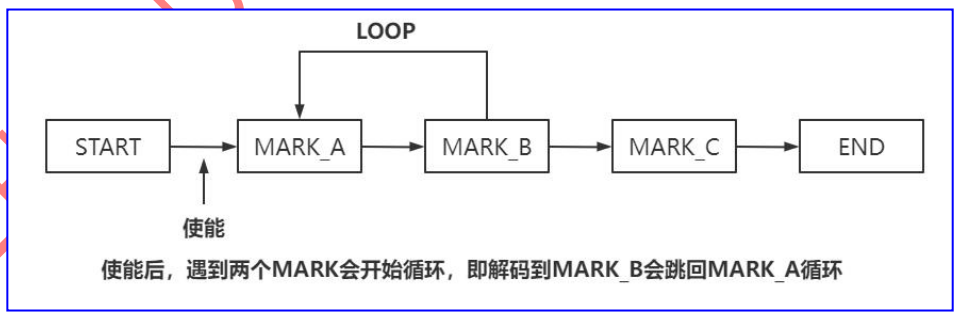
1.2.5.1 MARK 回调函数

初始化中 MARK 回调功能，并置上 MARK_ENABLE 使能位，解码中读取到乐谱中的 MARK 标记后会发生回调。回调函数 u32 midi_mark_trigger(void *priv, u8 *val, u8 len)的参数 val 是指向 MARK 名称的指针，len 是名称的长度。Sdk 默认配置如下：

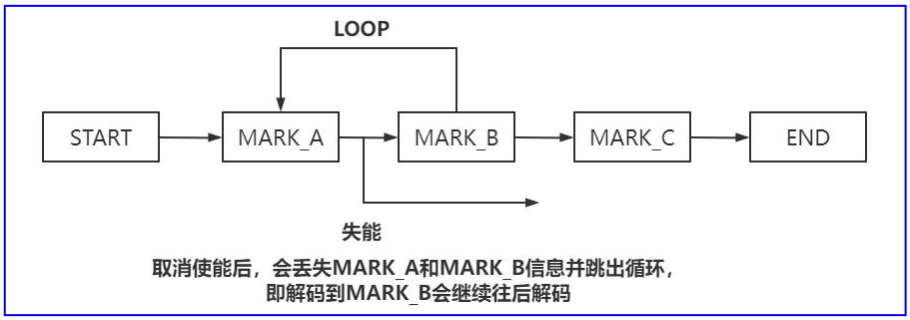


1.2.5.2 两个 MARK 之间循环播放

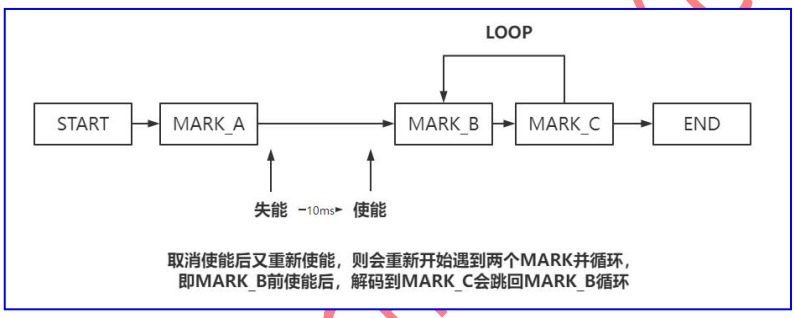
1）MARK 标记还可以实现两个 MARK 之间循环解码功能，置上 MARK_LOOP_ENABLE 使能位后，遇到两个 MARK 后会在它们之间循环播放，如下图：



2）在循环播放中取消 MARK_LOOP_ENABLE 使能会跳出循环，同时丢失两个 MARK 信息，如下图：



3）MARK_A 和 MARK_B 中间循环时取消使能后，在解码到 MARK_B 之前再次置上 MARK_LOOP_ENABLE 使能位后，会重复（1）的过程，遇到两个 MARK 后又会开始循环，取消使能到再次使能之间需要让解码器 run 至少 2~3 次，在主循环中调用一个 10ms 延时即可，如下图：



1.2.6 其他回调功能

1.2.6.1 主旋律的音符回调功能

置上 MELODY_ENABLE 使能位，解码到音符会回调：

u32 midi_melody_trigger(void *priv, u8 key, u8 vel)函数，其参数 priv 是私有参数，key 是音符，key 为 255 表示是静音回调，vel 是力度。

1.2.6.2 主旋律的音符结束回调功能

置上 MELODY_STOP_ENABLE 使能位，音符解码结束后会回调：

初始化配置 MIDI_INIT_STRUCT 结构体中 moledy_stop_info 回调时，需要配置回调模式 main_chn_enable，该配置仅在 midi 解码模式下生效，midi 琴模式无效，其中：

- main_chn_enable = 1，midi 解码音符结束仅回调主旋律的音符；
- main_chn_enable = 0，midi 解码音符结束会回调所有音符；

ZHUHAIJIELI TECH CO.,LTD

u32 midi_melody_stop_trigger(void *priv, u8 key, u8 chn)函数，其参数 priv 是私有参数，key 是音符，chn 是音符通道。

1.2.6.3 节拍(BEAT)的回调

置上 BEAT_TRIG_ENABLE 使能位，节拍来时会回调：

u32 midi_beat_trigger(void *priv, u8 val1, u8 val2)函数，其中参数 priv 是私有参数，val1 是一节多少拍，vel 是每拍多少分音符。

1.2.6.4 小节的回调

置上 TIM_DIV_ENABLE 使能位，小节开始会回调函数：

u32 midi_timeDiv_trigger(void *priv)函数，priv 是私有参数。

1.2.6.5 清狗的回调

清狗回调在初始化中注册即可，只有主旋 OKON 副旋不发声的 OKON 工作模式会使用该回调，详情请看《2.3.2OKON 的工作模式》说明。

1.2.7 外部音量功能（CMD_MIDI_SET_EX_VOL）

MIDI 解码能够实现每个通道/轨道单独调节音量的功能，该功能需要置上 EX_VOL_ENABLE 使能位。其结构体参数 EX_CH_VOL_PARM 包括：

① **cc_vol [CTRL_CHANNEL_NUM]**

16 个通道或者轨道的音量大小，是与音源音量比例关系的 12 bit 放大，1 代表使用音源音量，1<<12 得到 4096，cc_vol 值为 4096 则不修改音量；如增到到音源音量的 2 倍，则 2<<12 得到 8192，以此类推。外部音量不建议放大，会有破音的可能；

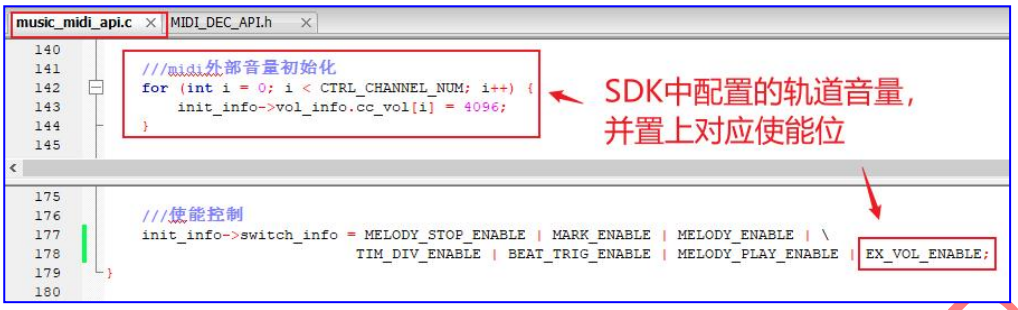
ex_vol_use_chn 参数决定修改的是通道，或者轨道音量；

② **ex_vol_use_chn**

音量通道选择，0 表示 cc_vol 修改的是轨道音量，1 表示 cc_vol 修改的是通道音量；

ZHUHAI JIELI TECHNOLOGY CO.,LTD

Sdk 默认配置如下：



该功能可以在 MIDI 运行中动态设置，运行中想要单独关闭某一通道/轨道的声音，可以将该通道/轨道音量设为 0。调用函数 music_midi_ctrl_config()，并将 CMD_MIDI_SET_EX_VOL 命令和配置好的结构体参数传入；置上 EX_CH_VOL_PARM 使能位即可。详情请看《2.1.3MIDI 乐谱解码的 CMD 配置》说明。

1.2.8 移半音功能（CMD_MIDI_SET_SEMITONE）

MIDI 解码能够实现每个通道单独偏移 n 个半音，该功能需要置上 SEMITONE_ENABLE 使能位。其结构体参数 MIDI_SEMITONE_CTRL_STRUCT 如下：

① **Key_diff[CTRL_CHANNEL_NUM]**

每个通道移动多少半音的配置；

SDK 中默认没有对该功能进行初始化，用户可自行添加。

该功能可以在 MIDI 运行中动态设置。调用函数 music_midi_ctrl_config()，并将 CMD_MIDI_SET_SEMITONE 命令和配置好的结构体参数传入；置上 SEMITONE_ENABLE 使能位即可。详情请看《2.1.3MIDI 乐谱解码的 CMD 配置》说明。

1.2.9 更改节奏和衰减（CMD_MIDI_CTRL_TEMPO）

MIDI 解码能够强制改变节奏、每个通道的衰减程度以及 mute 阈值水平，其结构体参数 MIDI_PROG_CTRL_STRUCT 如下：

① **tempo_val**

节奏速度，默认值为 1042；数值越大，节奏越慢；

ZHUHAI JIELI TECHNOLOGY CO.,LTD

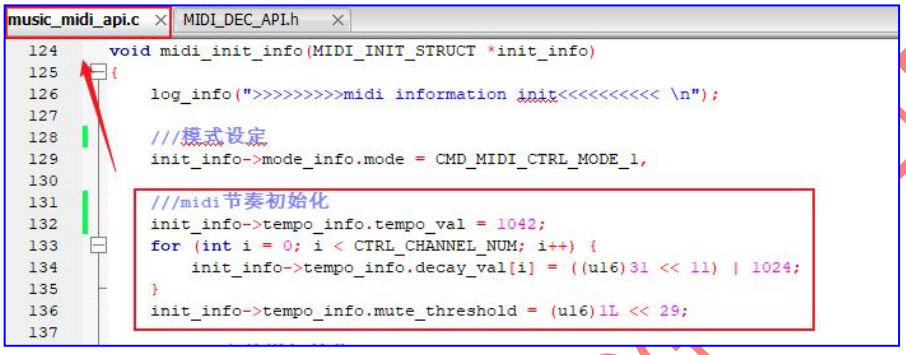
② **decay_val [CTRL_CHANNEL_NUM]**

衰减参数，低 11bit 有效，每个通道单独配置；数值越小，余音衰减越快；

③ **mute_threshold**

mute 阈值，数值越高，主旋的静音音符回调会越多，需要置上 MELODY_ENABLE 使能位。

Sdk 中的默认配置如下：



该功能可以在 MIDI 运行中动态设置。调用函数 music_midi_ctrl_config()，并将 CMD_MIDI_CTRL_TEMPO 命令和配置好的结构体参数传入即可。详情请看《2.1.3MIDI 乐谱解码的 CMD 配置》说明。

1.2.10 小节回退功能（CMD_MIDI_SEEK_BACK_N）

MIDI 运行中可以实现回退到之前的小节，需要在初始化中置上 SAVE_DIV_ENBALE 使能位。该功能不支持连续回退小节，例如回退了 8 个小节后，需要再次经过 8 个小节才能进行下一次回退，否则可能会导致小节混乱。其结构体参数 MIDI_SEEK_BACK_STRUCT 如下：

① **seek_back_n**

回退的小节数，范围为 1~8，即最大可回退 8 个小节；值为 1 时回退到当前小节起始位置；

该功能只能在 MIDI 运行时才能实现。初始化中置上 SAVE_DIV_ENBALE 使能位，调用函数 music_midi_ctrl_config()，并将 CMD_MIDI_SEEK_BACK_N 命令和配置好的结构体参数传入即可。详情请看《2.1.3MIDI 乐谱解码的 CMD 配置》说明。

ZHUHAI JIELI TECHNOLOGY CO.,LTD

1.2.11 MIDI 解码函数接口

1.2.11.1 函数 void midi_decode_init(void)

该函数实现查找 midi 解码的音色库并记录音色库信息，在系统初始化中调用；

1.2.11.2 函数 int music_midi_ctrl_config(music_play_obj *hdl, u32 cmd, void *parm)

该函数实现配置 midi 解码的功能参数，cmd 的具体介绍请看第 2 节 MIDI 乐谱解码介绍，其中参数：

1. hdl：midi 解码句柄；
2. cmd：命令，可用命令以及相应的参数结构体如下；
   ① CMD_MIDI_SEEK_BACK_N，参数结构体 MIDI_SEEK_BACK_STRUCT；
   ② CMD_MIDI_SET_CHN_PROG，参数结构体 MIDI_PROG_CTRL_STRUCT；
   ③ CMD_MIDI_CTRL_TEMPO，参数结构体 MIDI_PLAY_CTRL_TEMPO；
   ④ CMD_MIDI_GOON，无参数，传 NULL 即可；
   ⑤ CMD_MIDI_CTRL_MODE，参数结构体 MIDI_PLAY_CTRL_MODE；
   ⑥ CMD_MIDI_SET_SWITCH，参数为所有使能位；
   ⑦ CMD_MIDI_SET_EX_VOL，参数结构体 EX_CH_VOL_PARM；
   ⑧ CMD_INIT_CONFIG，参数结构体 MIDI_INIT_STRUCT；
   ⑨ CMD_INIT_CONFIGS，参数结构体 MIDI_CONFIG_PARM；
   ⑩ CMD_MIDI_OKON_MODE，参数结构体 MIDI_OKON_MODE；
   ⑪ CMD_MIDI_SET_SEMITONE，结构体 MIDI_SEMITONE_CTRL_STRUCT
3. parm：命令对应的参数结构体，请看 cmd 参数说明；
4. 返回值：成功 0；
   失败 其他值；

ZHUHAI JIELI TECHNOLOGY CO.,LTD

1.2.11.3 函数 void midi_okon_mode_set(music_play_obj *music_obj, u8 mode)

该函数实现配置 midi 解码的模式，其中参数：

1. music_obj：midi 解码句柄；
2. mode：midi 解码模式，可用 mode 如下：
   ① CMD_MIDI_CTRL_MODE_0，正常解码模式；
   ② CMD_MIDI_CTRL_MODE_1，one key one note 模式；
   ③ CMD_MIDI_CTRL_MODE_2，只推消息不出声模式；
   ④ CMD_MIDI_CTRL_MODE_W2S，外部音源模式；

1.2.11.4 函数 void midi_okon_goon(music_play_obj *music_obj)

该函数实现 midi 解码 okon 模式启动下一个音符，其中参数：

1. music_obj：midi 解码句柄；

1.2.11.5 函数 void midi_init_info(MIDI_INIT_STRUCT *init_info)

该函数实现 midi 解码配置初始化，会给 midi 解码器主动调用，其中参数：

1. init_info：midi 解码初始化配置结构体，具体请参照代码中的配置信息；

ZHUHAI JIELI TECHNOLOGY CO.,LTD

1.3 MIDI 琴

MIDI 琴功能支持通过函数驱动 midi 解码。该应用依赖我司自定义的 mid 音色库，用户可通过我司 mid 工具进行转换，详情请看《JLmidi 工具使用说明》。本节将介绍 midi 琴初始化过程和相关的接口函数。

1.3.1 MIDI 琴初始化

1.3.1.1 初始化过程

1) 在系统初始化时会调用 void midi_ctrl_decode_init(void)函数记录 MIDI 音色库文件的信息；midi 琴 MAX_CTR_PLAYER_CNT 同时发声的 key 数默认为 15。
2) 调用 decoder_init(void)初始化解码器；
3) 调用 decoder_io(void *pfile, u32 dec_ctl, dp_buff *dbuff, u8 loop)进行解码，该函数 dec_ctl 参数传入 BIT_MIDI_CTRL;
4) 配置各通道乐器，弹奏解码。

1.3.1.2 注意要点

1) 同时发声的最大通道数 MAX_PLAYER_CNT 可以在 midi_ctrl_api.c 中根据需要进行设置，范围是 1~32，该值的大小会影响到解码 buf 的大小，sdk 默认配置如下：

```
midi_ctrl_api.c
25  u16 obuf_midi_ctrl[DAC_DECODER_BUF_SIZE / 2] AT(.midi_ctrl_buf) ;
26  u32 midi_ctrl_decode_buff[4984 / 4] AT(.midi_ctrl_buf) ;
27  #define MIDI_CTRL_CAL_BUF ((void *)&midi_ctrl_decode_buff[0])
28  MIDI_CONFIG_PARM midi_ctrl_t_parm AT(.midi_ctrl_buf);
29  MIDI_CTRL_PARM midi_ctrl_parmt AT(.midi_ctrl_buf);
30
31  //midi琴midi琴最大发声的key数选择,这个值库里面会使用,决定了解码buffer的大小
32  const int MAX_PLAYER_CNT = 18;//决定同时发声可配置的最大数,范围[1,32],值大小会影响解码buf大小
33
```

2) SDK 中 MIDI 琴中的通道号 chn 符合 mid 标准，一共有 16 个通道（0~15）。

其中 0~8、10~15 号通道可以使用 0~127 号乐器，9 号通道用于 128 号以上的打击乐器。

如选择 00-01 高亮大钢琴，那么通道号可以选择 0~8 或 10~15，乐器号写 0*8+1 = 1；

如选择 16-00standard，乐器号为 16*8+0=128，大于 127，所以通道号只能选 9，乐器号写：16*8+0-128 = 0；

ZHUHAI JIELI TECHNOLOGY CO.,LTD

1.3.2 MIDI 琴音符回调功能

midi 琴运行时，弹奏音符后可回调响应函数；该功能与 midi 解码的主旋律音符回调一致；区别在于 midi 琴中的回调配置只能在运行中通过动态配置的方式使能，可参考《1.2.1.4MIDI 在运行中动态配置功能示例》以及 SDK 中的说明，配置顺序如下：

1）CMD_MIDI_MELODY_TRIGGER 配置音符开始的回调；

2）CMD_MIDI_STOP_MELODY_TRIGGER 配置音符结束的回调；

3）CMD_MIDI_SET_SWITCH 配置回调使能，switch_enable 使能参数需要置上 MELODY_ENABLE 和 MELODY_STOP_ENABLE 使能位；

1.3.3 MIDI 琴函数接口

1.3.3.1 函数 void midi_ctrl_decode_init(void)

该函数实现查找 midi 琴的音色库并记录音色库信息，在系统初始化中调用；

1.3.3.2 函数 int music_midi_ctrl_set_prog(music_play_obj *hdl, u8 prog, u8 chn)

该函数实现查找设置 midi 琴的乐器，0 其中参数：

1. hdl：midi 琴解码句柄；
2. prog：乐器号，与 midi 音色库制作工具中的乐器号一致。详情请看《1.3.1.2 注意事项》说明；
3. chn：发声通道，该值范围在 0~15；
4. 返回值：成功 0；
   失败 其他值；

1.3.3.3 函数 int music_midi_ctrl_pitch_bend(music_play_obj *hdl, u16 pitch_val, u8 chn)

该函数实现配置 midi 琴弯音轮参数，其中参数：

1. hdl：midi 琴解码句柄；

ZHUHAI JIELI TECHNOLOGY CO.,LTD

2. pitch_val：弯音值，默认值是 256，可根据该值调整；
3. chn：发声通道，该值范围在 0~15；
4. 返回值：成功 0；
   失败 其他值；

1.3.3.4 函数 int music_midi_ctrl_config(music_play_obj *hdl, u32 cmd, void *parm)

该函数实现配置 midi 琴的其他功能，其功能参数和 midi 解码中的一致，其中参数：

1. hdl：midi 琴解码句柄；
2. cmd：命令，可用命令如下；
   ① CMD_MIDI_CTRL_TEMPO；
   ② CMD_MIDI_SET_EX_VOL；
   ③ CMD_INIT_CONFIGS；
   ④ CMD_MIDI_SET_SEMITONE；
   ⑤ CMD_MIDI_SET_SWITCH；
3. parm：命令参数，可查看 midi 解码中的介绍；
4. 返回值：成功 0；
   失败 其他值；

1.3.3.5 函数 int music_midi_ctrl_note_on(music_play_obj *hdl, u8 nkey, u8 nvel, u8 chn)

该函数实现弹奏指定通道的音符，其中参数：

1. hdl：midi 琴解码句柄；
2. nkey：音符，与 midi 音色库制作工具中的音符号一致，范围在 0~127；
3. nvel：琴键按下力度，范围在 0~127；部分乐器（如钢琴）有效，不同力度会使按下后的余音长短不同；
4. chn：发声通道，该值范围在 0~15；
5. 返回值：成功 0；
   失败 其他值；

ZHUHAI JIELI TECHNOLOGY CO.,LTD

1.3.3.6 函数 int music_midi_ctrl_note_off(music_play_obj *hdl, u8 nkey, u8 chn, u16 decay_time)

该函数实现查找设置 midi 琴的乐器。对于声音不会衰弱的乐器（如笛子），调用 Note_on 函数后需要调用该函数以结束音符，对于声音会衰弱的乐器（如钢琴）可不用。其中参数：

1. hdl：midi 琴解码句柄；
2. nkey：音符，与 midi 音色库制作工具中的音符号一致，范围在 0~127；
3. chn：发声通道，该值范围在 0~15；
4. decay_time：衰减时间，单位 ms，若为 0 则使用音色自带的衰减；
5. 返回值：成功 0；
   失败 其他值；

1.3.3.7 函数 int music_midi_ctrl_vel_vibrate(music_play_obj *hdl, u8 nkey, u8 vel_step, u8 vel_rate, u8 chn)

该函数实现配置音符的颤音效果，其中参数：

1. hdl：midi 琴解码句柄；
2. nkey：音符，与 midi 音色库制作工具中的音符号一致，范围在 0~127；
3. vel_step：抖动步长，该值范围在 0~7；
4. vel_rate：抖动幅度，该值范围在 0~15；
5. 返回值：成功 0；
   失败 其他值；

1.3.3.8 函数 int midi_ctrl_play_end_deal(music_play_obj *hdl)

该函数实现停止 midi 琴解码，在所有音符停止后会被调用，其中参数：

1. hdl：midi 琴解码句柄；
2. 返回值：无；

ZHUHAI JIELI TECHNOLOGY CO.,LTD

1.3.3.9 函数 void midi_ctrl_callback_init(music_play_obj *hdl, u32(*melody_callback)(void *, u8, u8), u32(*melody_stop_callback)(void *, u8))

该函数实现配置音符开始与结束的回调函数，其中参数：

1. hdl：midi 琴解码句柄；
2. melody_callback：音符开始回调函数，回调函数的原型参照《1.2.6.1 主旋律的音符回调功能》中的说明；
1. melody_stop_callback：音符结束回调函数，回调函数的原型参照《1.2.6.2 主旋律的音符结束回调功能》中的说明；
2. 返回值：无；

1.3.3.10 函数 void get_midi_ctrl_play_key(music_play_obj *hdl, u8 chn)

该函数实现获取指定通道正在播放的所有 key 值，其中参数：

1. hdl：midi 琴解码句柄；
2. chn：需要查询的通道；
3. 返回值：无；

ZHUHAI JIELI TECHNOLOGY CO.,LTD

1.4 midi 限幅器

midi 声音削顶采用限幅器的方法进行处理，在 midi 库中需要回调限幅器的运行函数。

1.4.1. 使用说明

1、在打开 midi 琴或者 midi 解码前，先初始化限幅器

```
#if USER_CONFIG_LIMITER
u16 ctl_work_buf[60 / 2] AT(.midi_ctrl_buf);
#endif
void toy_midi_keyboard_app(void)
{
    log_info("toy midi keyboard mode!\n");
    int msg[2], err;
    u8 scale = 5;
    u8 key_offset = 0;
    u8 cur_channal = 0;
    u16 pitch_val = 256;
    midi_keyboard_idle_cnt = 0;
#if USER_CONFIG_LIMITER //limiter
    int bufsize = Limiter_QueryBufSize();
    log_info("bufsize %d\n", bufsize);
    int attacktime = 5;
    int releasetime = 300;
    int threshold = -40000;
    int samplerate = 22050;
    int channel = 1;
    Limiter_Init(ctl_work_buf, attacktime, releasetime, threshold, samplerate, channel);
#endif
```

限幅器运行buf

限幅器初始化

2、打开 midi 琴或者 midi 解码后，配置并使能 midi 限幅器回调。注意 ctl_limiter_info.limiter_buf 与 midi_buf（midi 琴或 midi 解码使用的 buf）不能是同一个。

```
static void midi_on_off_callback_init(dec_obj *obj, u32(*melody_callback)(void *, u8, u8), u32(*melody_stop_callback)(void *, u8)
{
    audio_decoder_ops *ops = (audio_decoder_ops *)obj->dec_ops;

    EX_MELODY_STRUCT melody_parm;
    melody_parm.priv = obj;
    melody_parm.melody_trigger = melody_callback;
    ops->dec_config(obj->p_dbuf, CMD_MIDI_MELODY_TRIGGER, &melody_parm);

    EX_MELODY_STOP_STRUCT melody_stop_parm;
    melody_stop_parm.priv = obj;
    melody_stop_parm.melody_stop_trigger = melody_stop_callback;
    ops->dec_config(obj->p_dbuf, CMD_MIDI_STOP_MELODY_TRIGGER, &melody_stop_parm);

    midi_key_board_function_switch |= MELODY_ENABLE | MELODY_STOP_ENABLE;
    ops->dec_config(obj->p_dbuf, CMD_MIDI_SET_SWITCH, &midi_key_board_function_switch);

#if USER_CONFIG_LIMITER //limiter
    MIDI_Limiter ctl_limiter_info;
    ctl_limiter_info.limiter_buf = ctl_work_buf;
    ctl_limiter_info.limiter_run = Limiter_Run;
    ops->dec_config(obj->p_dbuf, CMD_MIDI_LIMITER_TRIGGER, &ctl_limiter_info);
    midi_key_board_function_switch |= LIMITER_ENABLE;
    ops->dec_config(obj->p_dbuf, CMD_MIDI_SET_SWITCH, &midi_key_board_function_switch);
#endif
}

#endif
```

1.4.2. 参数说明

int bufsize = Limiter_QueryBufSize(); //获取限幅器运行 buf 大小

int attacktime; //限幅器启动时间，输入信号超过阈值启动限幅器所需要的时间

int releasetime; //限幅器释放时间，输入信号低于阈值恢复到原始信号所需要的时间

int threshold; //限幅器阈值，单位 mdb,范围(-90000,0),超过阈值启动限幅器

int samplerate; //采样率，midi 琴或者 midi 解码的采样率

int channel; //通道数，midi 琴或者 midi 解码的通道

第二章 MIDI 工具

2.1 MIDI 工具主界面

2.1.1 MIDI 版本说明

带有“压缩编辑”选项和“导入 SF2 音色库”选项的为新版工具（JLmidi_new_0706），只支持 AD14 和 AD15 在 SDK 版本大于等于 V1.3.0 的 SDK 上运行；

旧版本工具（JLmidi_60）支持所有版本 SDK。

新版和旧版工具均可在 gitee 上下载。

2.1.2 MIDI 工具简介

MIDI 文件是一种事件文件，不含音源，它记录的是一系列的事件。

MIDI 文件解码，是从 MIDI 音源库中查找（或者根据一定的算法计算）得到事件对应的音频，因此对于不同的音源库而言，相同的 MIDI 文件解码出来的声音不同。

我司 MIDI 音源文件不会记录每个按键的声音，解码时再依次查找叠加，这种做法会导致音源库很大。我司的做法是：假设 0 号乐器的第 n 到 m 号按键是可以通过 N 号按键计算的（m<=(N+12)），那么库中只需要记录 N 号按键。比方说，0-20 号 key 都用 10 号 key 的声音根据频率关系去计算，21-47 号 key 都用 48 号 key 去计算…那么只需记下 0 号乐器的第 10、48…号按键。

本工具的作用：可以查看，某个乐器某个按键的音源及其信息。也可以进行编辑修改成自己想要的声音。

点击“pWM.exe”，打开 midi 工具。

ZHUHAI JIELI TECHNOLOGY CO.,LTD



2.1.3 主界面-乐器种类

图中的红色框为乐器大类，共 32 项；蓝色框为乐器细分种类，共 8 项。对应的乐器号计算方法：假设乐器种类为 0-31，乐器细分为 0-7。乐器号为（乐器种类*8+乐器细分）。

在“打击乐器”中，有很多的乐器是没有的，用户可以自主添加乐器声。

鼠标点击“乐器细分种类”后，可点击按键试听音色。

2.1.4 主界面-选中乐器

在 midi 文件转换的时候，工具会自动把文件中所使用的乐器加载到库中。同时，用户也可以手动添加一些乐器（用于 midi 琴等），在 midi 转换的时候把这些库也加载到库中。选择乐器时会提示选择乐器的哪些 key，未选择的 key 在 SDK 弹奏的结果不可预测。

SDK 程序中 midi 琴有 16 个通道，其中 0~8 和 9~15 号通道可以使用 0-127 号乐器，9 号通道只能使用 128-255 号乐器。

如选择 00-01 高亮大钢琴，那么通道号可以选择 0~8 或 10~15，乐器号写 0*8+1 = 1；

如选择 16-00 standard，乐器号为 16*8+0=128，大于 127，所以通道号只能选 9，乐器号写：16*8+0-128 = 0；

ZHUHAI JIELI TECHNOLOGY

2.2 MIDI 转换

midi 转换过程包括选择输出目录、转换和导出库三个步骤。

原始的 midi 文件需要通过我司的 MIDI 转换工具“转换”后才能正常使用；“导出库”会将 midi 文件中用到的乐器制作成音色库。

若是只用到 midi 琴的应用，在主界面选择自己所需要的乐器后，直接点击“导出库”选项即可生成相应的音色库。

2.2.1 选择需要转换的 MIDI 文件

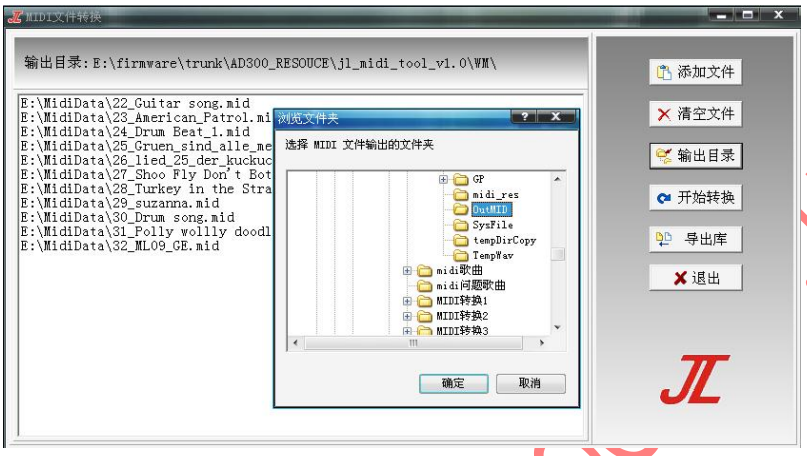
在 MIDI 工具主界面中点击“MID 转换”弹出 MIDI 文件转换工具，点击“添加文件”按钮，在弹出的文件选择框中选中要转换的一个或多个 midi 文件。



ZHUHAI JIELI TECHNOLOGY CO.,LTD

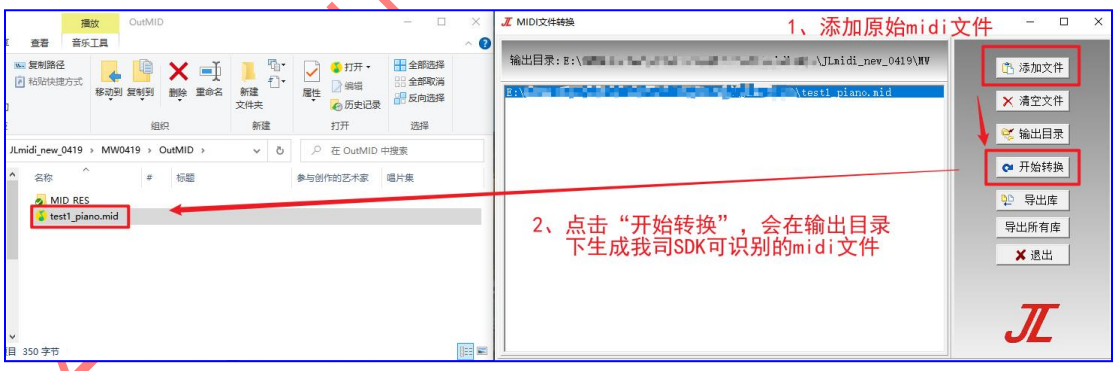
2.2.2 输出目录

点击“输出目录”，选择转换后 midi 文件存放的文件夹。默认路径为“*工具根目录\OutMID*”文件夹中。



2.2.3 转换

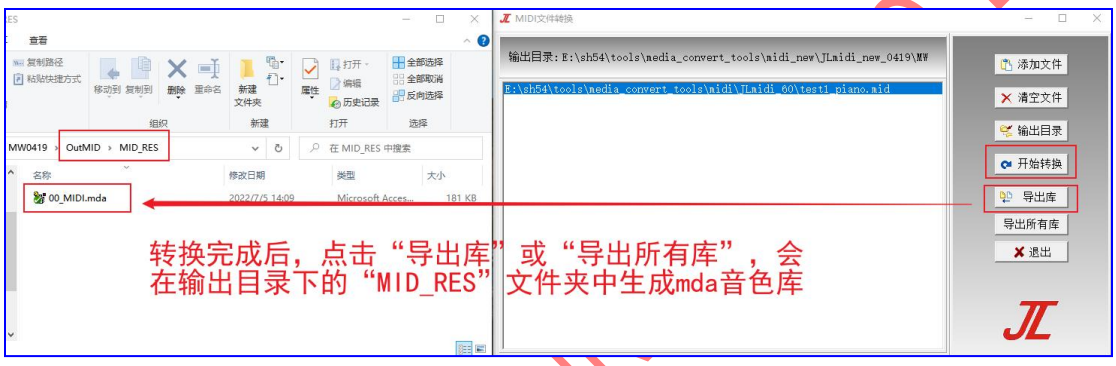
点击“开始转换”，即可在设置的输出目录下生成我司自定义的 mid 文件，右下角显示转换进度。此时按键灰掉不能使用，等待转换完成才恢复。



2.2.4 导出库

转换完成后，点击“导出库”，工具根据“输出目录”中的文件生成一个我司自定义的 mid 音色库文件，保存在“输出目录”的“MID_RES”文件夹中，默认为路径为“*工具根目录\OutMID\MID_RES*”文件夹中。

如果在主界面中有选中乐器，“导出库”会将 midi 文件需要的乐器和主界面选中的乐器一起打包制作成音色库文件。若是只用到 midi 琴的应用，在主界面选择自己所需要的乐器后，直接点击“导出库”选项即可生成相应的音色库。



注：新的 SDK 支持新版工具生成的“.mda”后缀音色库，且兼容旧版工具生成的“.mid”后缀音色库；旧的 SDK 只支持旧版工具生成的“.mid”后缀音色库。可通过 SDK 中 midi 初始化的代码判断是否支持新版工具制作的音色库。

```
midi_api.c
141  void midi_decode_init(void)
142  {
143      void *pvfs = 0;
144      void *pvfile = 0;
145      u32 err = 0;
146
147      err = vfs_mount(&pvfs, (void *)NULL, (void *)NULL);
148      if (err != 0) {
149          return;
150      }
151
152      err = vfs_openbypath(pvfs, &pvfile, "/midi_cfg/00_MIDI.mda");
153      if (err != 0) {
154          log_info("midi dec mda open fail, try old midi_cfg.bin!\n");
155          err = vfs_openbypath(pvfs, &pvfile, "/midi_cfg/midi_cfg.bin");
156          if (err != 0) {
157              vfs_fs_close(&pvfs);
158              return;
159          }
160      }
```

新版SDK支持带压缩的mda音色库
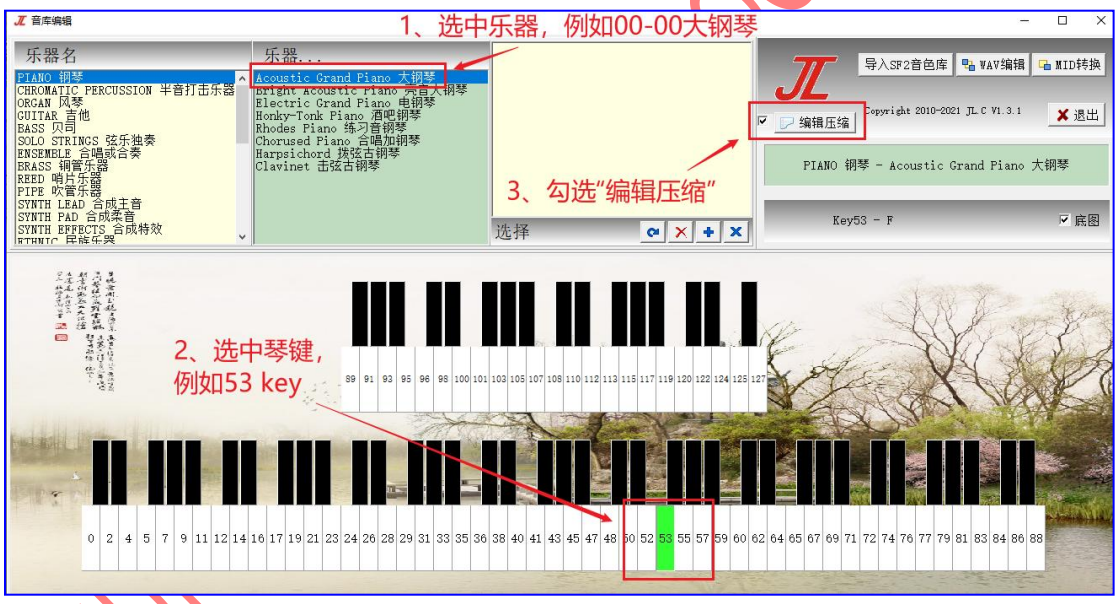
2.3 MIDI 音色库压缩

MIDI 一个乐器的相邻的一连串音符（key），并非都是单独的音频文件，可能是由同一个音源文件经过音频算法偏移得到的。

MIDI 音色库压缩就是压缩这类音源文件，因此压缩一个音源文件会影响到一连串音符（key）的音质，甚至会影响到同一个大类下的其他乐器的一连串音符的音质。

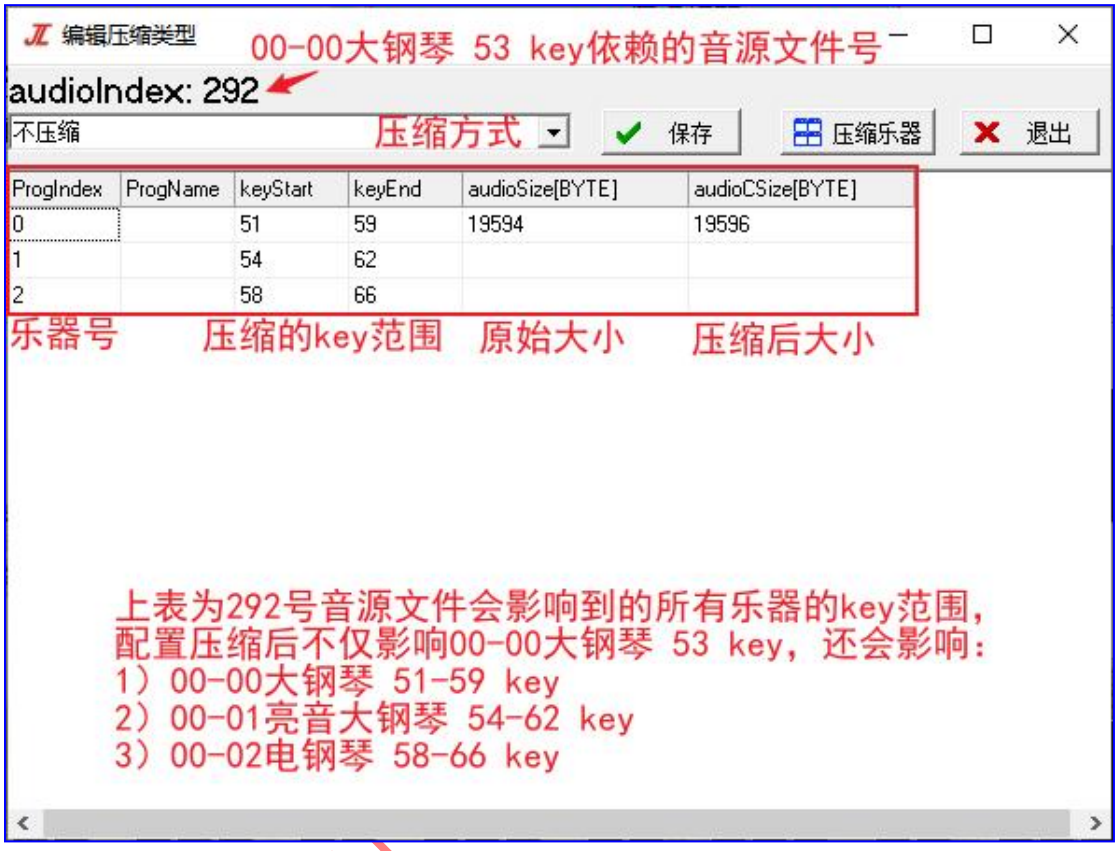
2.3.1 压缩流程

1) 鼠标点击蓝色框乐器细分种类和琴键后，可以通过点击右边“编辑压缩”选项，选择是否压缩对应的音色文件。



2) 选中相应的乐器，勾选并点击“编辑压缩”选项，会打开如下编辑界面，选择需要的压缩方式并保存；如果全部音源都是不压缩的，建议不要勾选该选项，因为压缩会带来额外开销。

3) 编辑界面会显示所选乐器的 key 的音源文件号，以及该音源文件会影响到的所有乐器及该乐器的 key 范围。改了压缩选项后，返回主界面弹奏对应的琴键，即可试听压缩后的效果。



4) 进入编辑界面算好压缩方式后，点击“压缩乐器”，会将所选乐器的所有音符对应的音源文件，全部按照所选压缩方式进行压缩。



5) 将该乐器压缩后的音色库导出，导出后可以查看更多乐器的压缩信息。



ZHUHAI JIELI TECHNOLO

2.3.2 压缩信息查询

导出库后，midi 工具根目录下会更新 ***midiinfo.html***、***progaudio.html***、***progaudio_all.html***、***proginfo.html*** 四个文件，这些文件能显示更加详细的音色和压缩信息。

1) ***midiinfo.html***

该文件可以查看转换的 midi 乐谱文件用到哪些乐器的哪些 key，可根据这里的信息再去压缩对应的音源文件。

JL MIDI INFO

MIDIFILE 为输入的midi文件列表
progIndex 为输入文件包含的音色序号
progName 为输入文件包含的音色名称
keyRange 为输入文件在该音色上有事件的key的范围
keyNum,loopMode,WavPos 为输入文件在该音色上使用到的音源信息
ProgSize :为输入文件在该音色上所对应的音源大小
ProgSize :为输入文件所包含的几种音色的音源大小综合
注意:可能会出现有几组key是用同一个keyNum及WavPos代表它们的音源波形是复用的，不会重复占用存储空间，但是它们的调制信息不一致

MIDIFILE	MIDIInfo								totalSize[BYTE]
	ProgIndex	ProgName	keyRange	keyNum	loopMode	WavLEN(short)	WavPos	ProgSize[byte]	
MIDI文件	乐器号	乐器名	key范围	48	1	11168	54194		
				56	1	9797	76530		
E:\sh55\download\MW0413\GP\midi_file_out\aizaixiyuan.mid	0	1 Acoustic Grand Piano 大钢琴	48-82	64	1	10516	96124	108358	108358
				73	1	7588	117156		
				78	1	7669	132332		
				84	1	7345	147670		
				41	1	13103	27988		
				48	1	11168	54194		
				56	1	9797	76530		
E:\sh55\download\MW0413\GP\midi_file_out\daoxiang.mid	0	1 Acoustic Grand Piano 大钢琴	40-88	64	1	10516	96124	134596	134596
				73	1	7588	117156		
				78	1	7669	132332		
				84	1	7345	147670		
				36	1	13672	644		
				41	1	13103	27988		
				48	1	11168	54194		
E:\sh55\download\MW0413\GP\midi_file_out\jiandanai.mid	0	1 Acoustic Grand Piano 大钢琴	28-81	56	1	9797	76530	147250	147250
				64	1	10516	96124		
				73	1	7588	117156		
				78	1	7669	132332		

2) ***progaudio.html***

该文件可以查看导出的音色库中的音源文件，是否哪种压缩算法，音源文件影响的乐器的 key 范围以及压缩前后的大小。其中：

- 工具默认的乐器所依赖的音源从 0 开始编号
- 用户自行导入的 SF2 音色库从 32768 开始编号
- 用户使用”WAV 编辑”生成的音色从 65536 开始编号

JL AUDIO_PROG INFO：显示该音源用在哪些乐器上

audioIndex:音源序号，用它来配置压缩方式
audioCOMPRESS:该音源使用的压缩算法
progIndex:乐器序号
progName: 乐器名称

audioIndex	audioCOMPRESS	audioInfo				audioSize[BYTE]	audioCSize[BYTE]
		ProgIndex	ProgName	keyStart	KeyEnd		
289	0	0	1 Acoustic Grand Piano 大钢琴	0	38	27344	27346
290	0	0	1 Acoustic Grand Piano 大钢琴	39	43	26206	26208
291	0	0	1 Acoustic Grand Piano 大钢琴	44	50	22336	22338
292	0	0	1 Acoustic Grand Piano 大钢琴	51	59	19594	19596
293	0	0	1 Acoustic Grand Piano 大钢琴	60	67	21032	21034
294	0	0	1 Acoustic Grand Piano 大钢琴	68	74	15176	15178
295	0	0	1 Acoustic Grand Piano 大钢琴	75	81	15338	15340
296	0	0	1 Acoustic Grand Piano 大钢琴	82	88	14690	14692
297	0	0	1 Acoustic Grand Piano 大钢琴	89	97	12884	12886
298	0	0	1 Acoustic Grand Piano 大钢琴	98	127	9684	9686

3) ***progaudio_all.html***

该文件与 progaudio.html 类似，区别在可以查看所有的音源文件，以及音源文件影响的乐器的 key 范围。

4) ***proginfo.html***

该文件可以查看导出库中包含的乐器信息，包括 key 映射的范围。

JL PROG INFO :显示该乐器用到哪些音源

TOTAL SIZE: 372066 BYTE
progIndex:乐器序号
progName: 乐器名称
progInfo: 乐器信息 [keyNum是存储的key的音高，start跟end是映射范围], loopMode代表是否为循环波形, WavLEN为波形长度,LoopLEN为波形循环部分长度, WavPOS代表波形地址
注意:可能会出现有几组key是用同一个keyNum及WavPos代表它们的音源波形是复用的，不会重复占用存储空间，但是它们的调制信息不一致

progIndex	progName	progInfo							progSize[无压缩情况下BYTE]
		keyNum	keyStart	keyEnd	loopMode	WavLEN(short)	LoopLEN(short)	WavPOS	
0	1 Acoustic Grand Piano 大钢琴	36	28	38	1	13672	1682	1774	161972
		41	40	43	1	13103	2272	29118	
		48	44	50	1	11168	1854	55324	
		56	51	59	1	9797	1487	77660	
		64	60	67	1	10516	1339	97254	
		73	68	74	1	7588	1393	118286	
		78	75	81	1	7669	1221	133462	
		84	82	88	1	7345	1368	148800	
7	8 Clavinet 击弦古钢琴	31	0	31	1	9341	902	163490	187078
		36	32	36	1	9491	671	182172	
		43	37	43	1	7907	450	201154	
		48	44	48	1	9273	337	216968	
		55	49	55	1	8595	224	235514	
		60	56	60	1	7625	168	252704	
		67	61	67	1	7679	225	267954	
		72	68	72	1	7547	84	283312	
		79	73	79	1	9280	225	298406	
		84	80	84	1	7737	295	316966	
		91	85	108	1	8645	253	332440	
		103	109	127	1	227	14	1320	
129	打击乐器	47	0	127	0	11168	11167	349730	22368

2.3.3 还原压缩配置内容

配置完压缩后，若需要重置压缩选项，可以删除工具根目录下的“config.txt”文件，再重启软件；

2.4 MIDI 导入 SF2 音色库

midi 工具除了可以使用我司默认的音色库外，还可以导入 SF2 音色库。

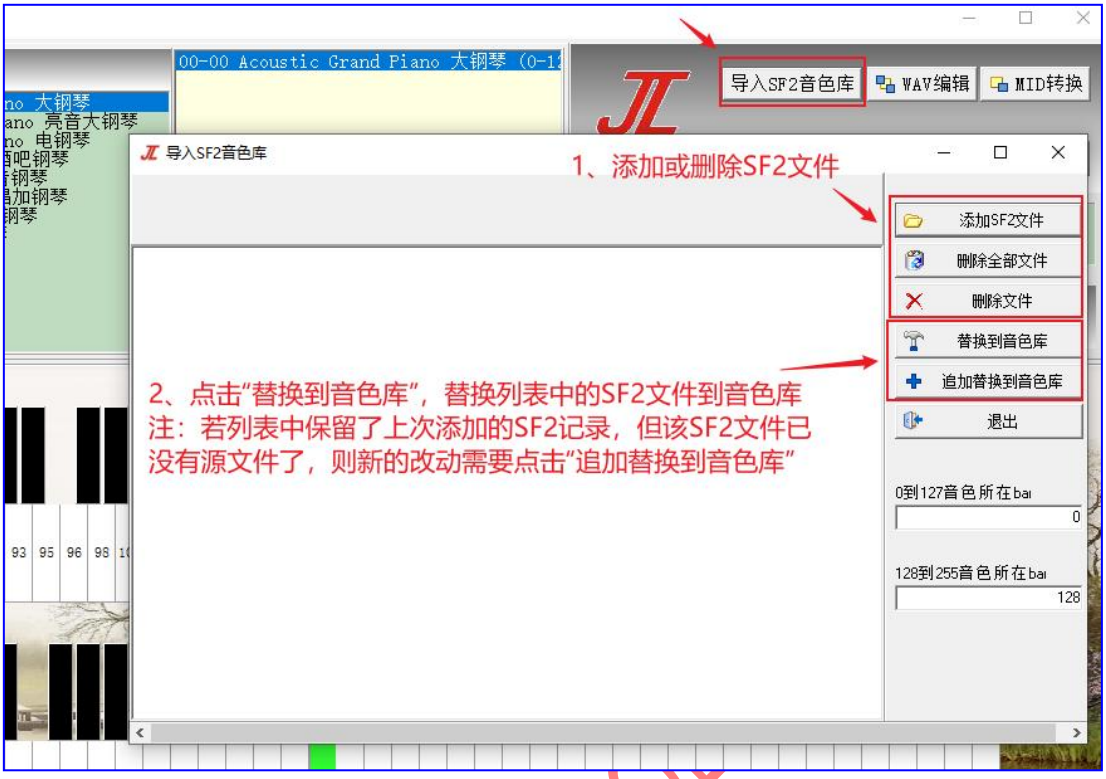
2.4.1 添加或删除 SF2 文件

主界面中点击“导入 SF2 音色库”选项，点击“添加 SF2 文件”选择 SF2 文件；单个 SF2 文件大小不可超过 65535 字节，否则会报错；

导入后点击“替换到音色库”。“替换到音色库”会删掉之前替换生成的 bin 文件，重新生成新的 bin 文件；如果列表中保留了上次添加的 SF2 记录，但 SF2 源文件已不存在了，则需要选择“追加替换到音色库”。替换完成后即可在主界面试听效果。

删除 SF2 库之后，需要重启软件才会生效。

ZHUHAI JIELI TECHNOLOGY CO.,LTD



ZHUHAI JIELI TECHNOLOGY

2.4.2 SF2 音色库信息查询

SF2 音色替换完成并将 MIDI 音色库导出后，可以在工具根目录下查看 ***progaudio.html*** 文件；所替换乐器的音源号大于 32768 小于 65535 即表示导出的音色库中 SF2 音色替换成功。

JL AUDIO_PROG INFO：显示该音源用在哪些乐器上

audioIndex:音源序号，用它来配置压缩方式
audioCOMPRESS:该音源使用的压缩算法
progIndex:乐器序号
progName: 乐器名称

audioIndex	audioCOMPRESS	audioInfo				audioSize[BYTE]	audioCSize[BYTE]
		ProgIndex	ProgName	keyStart	KeyEnd		
152	0	16	17 Hammond Organ 击杆风琴	109	127	290	0
289	0	0	1 Acoustic Grand Piano 大钢琴	28	38	27344	0
290	0	0	1 Acoustic Grand Piano 大钢琴	40	43	26206	0
291	0	0	1 Acoustic Grand Piano 大钢琴	44	50	22336	0
292	0	0	1 Acoustic Grand Piano 大钢琴	51	59	19594	0
293	0	0	1 Acoustic Grand Piano 大钢琴	60	67	21032	0
294	0	0	1 Acoustic Grand Piano 大钢琴	68	74	15176	0
295	0	0	1 Acoustic Grand Piano 大钢琴	75	81	15338	0
296	0	0	1 Acoustic Grand Piano 大钢琴	82	88	14690	0
32779	0	16	17 Hammond Organ 击杆风琴	0	72	1100	0
32780	0	16	17 Hammond Organ 击杆风琴	73	91	1100	0
32781	0	16	17 Hammond Organ 击杆风琴	92	104	214	0
32782	0	16	17 Hammond Organ 击杆风琴	105	108	156	0

2.4.3 还原 SF2 音色库内容

导入 SF2 音色库后，若需要重置 SF2音色库选项，可以删除工具根目录 GP 文件夹下的“sf2_config.bin”文件，再重启软件；

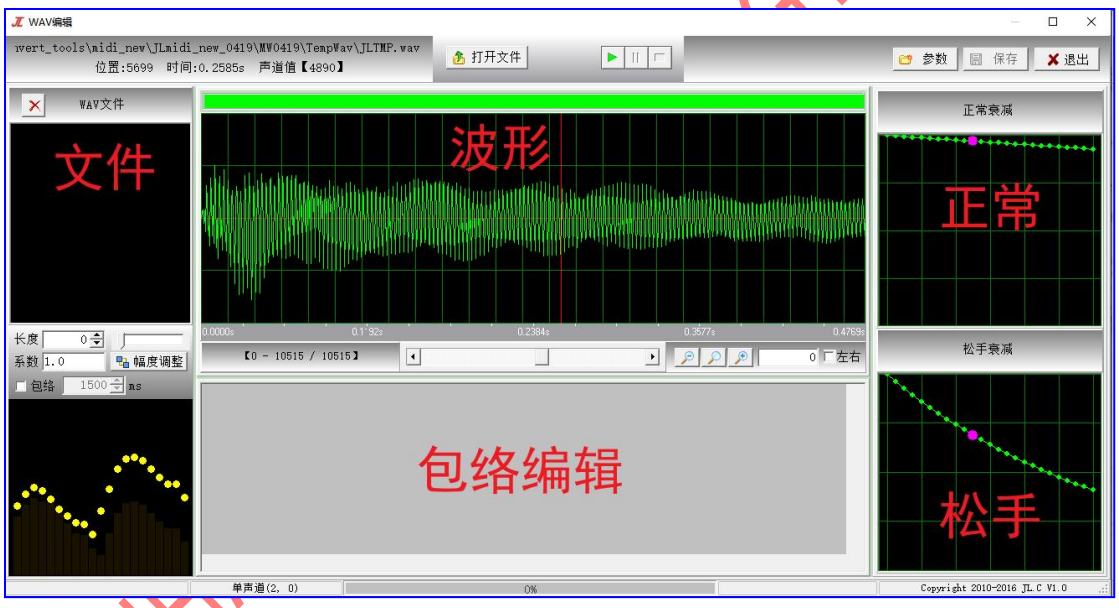
ZHUHAI JIELI TECHNOLOGY

2.5 MIDI WAV 编辑

2.5.1 WAV 编辑界面简介

在主界面中点击“WAV 编辑”进入 MIDI 音源编辑界面，编辑音源时记得要把乐器跟 key 都选好。

- 文件界面：当前选中的音频文件，可以点击“打开文件”添加自己的文件
- 波形界面：选中文件的 wav 波形显示与编辑
- 正常/松手衰减：音频的衰减情况
- 包络编辑界面：勾选左侧“包络”选项，可编辑当前文件 wav 波形的包络效果



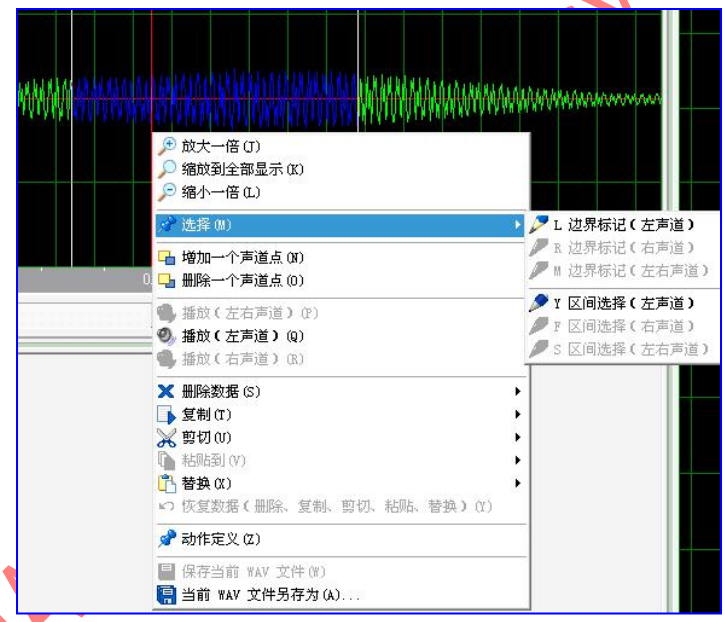
ZHUHAI JIELI TECHNOLOGY CO.,LTD

2.5.2 波形编辑

在波形图界面中点击右键弹出波形编辑框，点击“选择->边界标记”，再点击“选择->区间选择”选中一段区域，那么就可以对这一段音频进行复制\剪切之类的编辑了。

选择右击菜单的最后一项“当前 WAV 文件另存为…”，可以将编辑结果另存为自己的 wav 文件。否则编辑无效，就算是打开的默认音色的 wav 文件，也是生成的一个临时文件而已。如果不另存，下一次编辑也是会被改写的。

另外，提示打不开 tmp.wav，可能是库里面不包含该按键的声音。也可能没有选择 key，导致没有文件生成。具体检查方法，可以看下“.\GP”中 tmp.wav 的 Wav 文件存在，但是长度为 0，则该声音不存在。



ZHUHAI JIELI TECHNOLOGY CO.,LTD

2.5.3 参数配置

在“WAV 编辑”界面中点击“参数”进入参数编辑界面。如下图，图中是一个音色文件的单元信息，也就是该乐器的 key 码对应 wav 波形。音量、上升速度、正常衰减、松手衰减、衰减限制都是 0-1，但是工具中，以 65535 量化。所以，65535 对应 1，其中参数：

- Key 码：

  当前编辑的为乐器的 115 号按键

- KEY 范围：

  当前乐器的 115-126 号按键通过映射而来，映射范围在 0~127 之间，如果设置了非法值，则本次编辑无效

- 音量：

  默认音量的大小。可以调节该值，改变该乐器的默认音量

- 上升速度：

  改变的是一个按键发声的时候，它的淡入速度

- 正常衰减与松手衰减：

  该按键音的衰减速度。一般松手之后，声音会更快地衰减。右图曲线窗显示的只有前面 32ms 的衰减形状

- 衰减限制：

  按键长按时，声音会 hold 住的一个阈值。只有在松手后，声音才会低于该阈值继续衰减

- 循环起始位置：

  只对应循环有效的音源信息文件有效，即后面的小框是勾起的。简单的来说，如果不是循环的，那先忽略音量，就是一个音源文件有多长，那发声最长也就那样了。一个声音发完了就是发完了。但是，如果一个循环的音源文件，就是一个无穷长的文件，从循环位置开始一直到波形文件最后的这段波形视为一个周期，这个周期一直在现有的波形文件后面做周期性扩展。所以编辑的时候请注意，波形前后扩展时的过度，例如波形为[1,2,3,4,5,6,7]，3 为循环起始，那波形实则为[1,2,3,4,5,6,7,3,4,5,6,7,3,4...]。这时候，请保证，周期扩展连接处，即 7,3 之间。[1,2,3,4,5,6,7,3,4,5,6,7,3,4...]这个才是最后的波形，如果，这个声音在连接处不平滑，有啪一声，那就会一直啪啪响的了。例如波形是

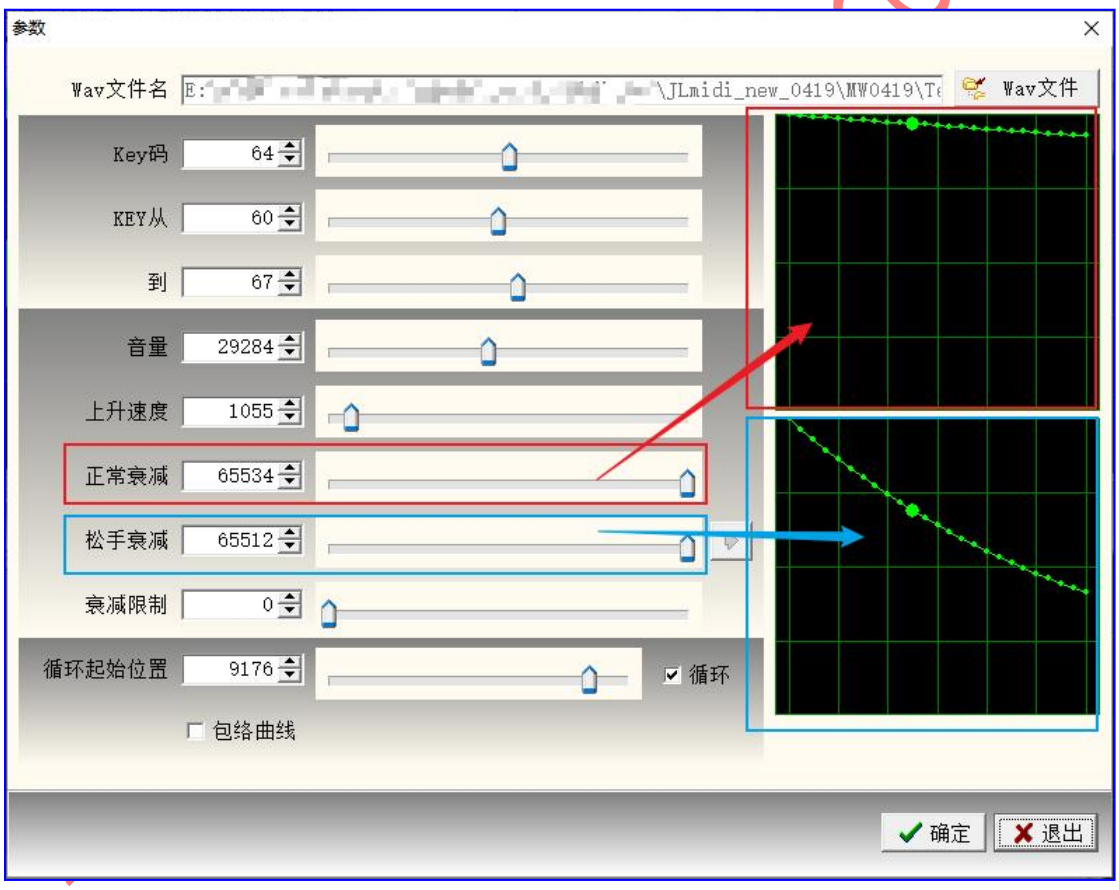
ZHUHAI JIELI TECHNOLOGY CO.,LTD

[1,2,3,4,5,6,5,4,3,2],则为[1,2,3,4,5,6,5,4,3,2, 3,4,5,6,5,4,3,2, 3,4,…]，要保证这个声音听起来好听。

- 包络曲线：

  选择是否需要编辑包络效果，与主界面“包络”勾选项一致。

另外需要确认的是，该窗口最上面的 wav 文件路径。请选择自己之前保存的 wav 文件路径，不要用临时路径。就是默认打开的音色编辑的 wav。该 wav 文件，是从库里剔出来生成的临时文件而已。下次是会被重写的。每次修改完保存的都只是路径而已。如果该 wav 无效了，或者被修改了。那都是按最后 wav 文件的内容进行库编辑的。



2.5.4 包络编辑

勾选主界面左侧的“包络”选项，或者在参数配置中勾选“包络曲线”，即可在主界面编辑 wav 音频的包络效果。

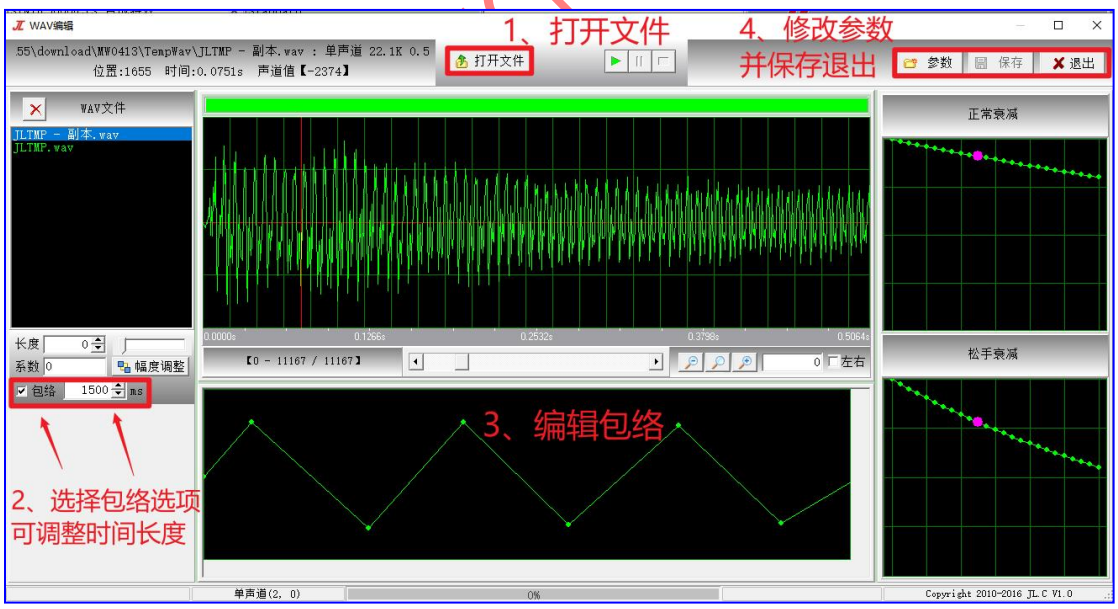
包络界面说明：

1) 纵轴代表幅值，范围为 0~100%；
2) 横轴代表时间，默认范围为 0~1500ms；可修改默认的 1500ms 时间长度，超过 wav 文件时间范围的话，表示会保持包络最后一点的能力；

包络编辑方法：

1) 双击增加包络点；
2) 长按可以拖动包络点；
3) 鼠标挪到包络点位置，点击右键即可删除节点；

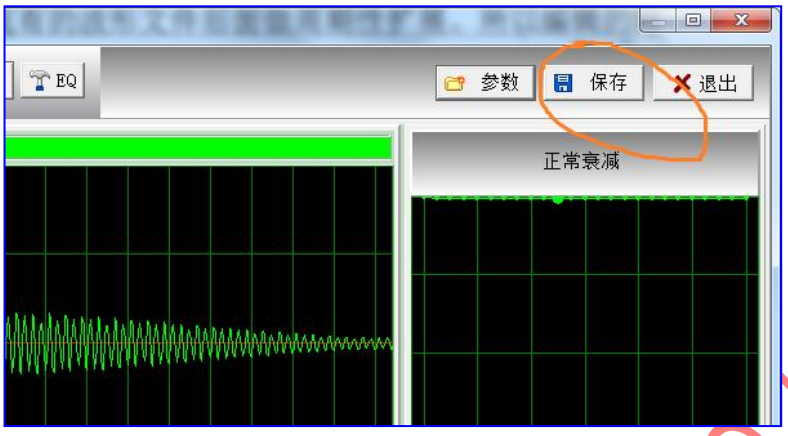
注意：使用包络模式，参数配置中的上升速度跟松手衰减这 2 个值依旧有效！



2.5.5 波形保存

修改完之后，点击确定。回到上级窗口，这时候保存按键亮起。如果点击保存，所做的修改就会被写入一个.bin 文件。否则无效。在退出的时候，会调用一个批处理去扫描.bin 文

件，生成最后需要的库。



2.5.6 WAV 编辑信息查询

WAV 编辑完成并将 MIDI 音色库导出后，可以在工具根目录下查看 ***progaudio.html*** 文件；所替换乐器的音源号大于 65535 即表示导出的音色库中 WAV 编辑成功替换了原始音源。

JL AUDIO_PROG INFO：显示该音源用在哪些乐器上

audioIndex:音源序号，用它来配置压缩方式
audioCOMPRESS:该音源使用的压缩算法
progIndex:乐器序号
progName: 乐器名称

audioIndex	audioCOMPRESS	audioInfo				audioSize[BYTE]	audioCSize[BYTE]
		ProgIndex	ProgName	keyStart	KeyEnd		
289	0	0	1 Acoustic Grand Piano 大钢琴	0	38	27344	27346
290	0	0	1 Acoustic Grand Piano 大钢琴	39	43	26206	26208
291	0	0	1 Acoustic Grand Piano 大钢琴	44	50	22336	22338
292	0	0	1 Acoustic Grand Piano 大钢琴	51	59	19594	19596
293	0	0	1 Acoustic Grand Piano 大钢琴	60	67	21032	21034
294	0	0	1 Acoustic Grand Piano 大钢琴	68	74	15176	15178
295	0	0	1 Acoustic Grand Piano 大钢琴	75	81	15338	15340
296	0	0	1 Acoustic Grand Piano 大钢琴	82	88	14690	14692
297	0	0	1 Acoustic Grand Piano 大钢琴	89	97	12884	12886
298	0	0	1 Acoustic Grand Piano 大钢琴	98	127	9684	9686
65536	0	0	1 Acoustic Grand Piano 大钢琴	56	56	19594	19596

2.5.7 还原 WAV 编辑内容

WAV 编辑完成后，若需要重置 WAV 编辑内容，可以删除工具根目录 GP 文件夹下的“config_out.bin”文件，再重启软件。

2.6 按键与乐器表

2.6.1 按键表

下表列出的是与音符相对应的命令标记。

八度音阶||　　音符号

	C	C#	D	D#	E	F	F#	G	G#	A	A#	B
0	0	1	2	3	4	5	6	7	8	9	10	11
1	12	13	14	15	16	17	18	19	20	21	22	23
2	24	25	26	27	28	29	30	31	32	33	34	35
3	36	37	38	39	40	41	42	43	44	45	46	47
4	48	49	50	51	52	53	54	55	56	57	58	59
5	60	61	62	63	64	65	66	67	68	69	70	71
6	72	73	74	75	76	77	78	79	80	81	82	83
7	84	85	86	87	88	89	90	91	92	93	94	95
8	96	97	98	99	100	101	102	103	104	105	106	107
9	108	109	110	111	112	113	114	115	116	117	118	119
10	120	121	122	123	124	125	126	127				

2.6.2 乐器表

乐器表可参考 SDK 中的 ***app\src\app_mg\midi\midi_prog.h*** 文件。

ZHUHAI JIELI TECHNOLOGY CO.,LTD

midi_prog.h

```
#ifndef __MIDI_PROG_H__
#define __MIDI_PROG_H__

#define  MIDI_PROG_ACOUSTIC_GRAND_PIANO                    0   //大钢琴
#define  MIDI_PROG_Bright_Acoustic_Piano                   1   //明音大钢琴
#define  MIDI_PROG_ELECTRIC_GRAND_PIANO                    2   //电钢琴
#define  MIDI_PROG_HONKY_TONK_PIANO                        3   //酒吧钢琴
#define  MIDI_PROG_RHODES_PIANO_PIANO                      4   //练习音钢琴
#define  MIDI_PROG_CHORUSED_PIANO                          5   //合唱加钢琴
#define  MIDI_PROG_HARPSICHORD_PIANO                       6   //拨弦古钢琴
#define  MIDI_PROG_CLAVINET_PIANO                          7   //击弦古钢琴
#define  MIDI_PROG_CELESTA_CHROMATIC_PERCUSSION            8   //钢片琴
#define  MIDI_PROG_GLOCKENSPIEL_CHROMATIC_PERCUSSION       9   //钟琴
#define  MIDI_PROG_MUSIC_BOX_CHROMATIC_PERCUSSION          10  //八音盒
#define  MIDI_PROG_VIBRAPHONE_CHROMATIC_PERCUSSION         11  //电颤琴
#define  MIDI_PROG_MARIMBA_CHROMATIC_PERCUSSION            12  //马林巴
#define  MIDI_PROG_XYLOPHONE_CHROMATIC_PERCUSSION          13  //木琴
#define  MIDI_PROG_TUBULAR_BELLS_CHROMATIC_PERCUSSION      14  //管钟
#define  MIDI_PROG_DULCIMER_CHROMATIC_PERCUSSION           15  //扬琴
#define  MIDI_PROG_HAMMOND_ORGAN                           16  //击杆风琴
#define  MIDI_PROG_PERCUSSIVE_ORGAN                        17  //打击型风琴
#define  MIDI_PROG_ROCK_ORGAN                              18  //摇滚风琴
#define  MIDI_PROG_CHURCH_ORGAN                            19  //管风琴
#define  MIDI_PROG_REED_ORGAN                              20  //簧风琴
#define  MIDI_PROG_ACCORDION_ORGAN                         21  //手风琴
#define  MIDI_PROG_HARMONICA_ORGAN                         22  //口琴
#define  MIDI_PROG_TANGO_ACCORDIAN_ORGAN                   23  //探戈手风琴
#define  MIDI_PROG_NYLON_ACOUSTIC_GUITAR                   24  //尼龙弦吉他
#define  MIDI_PROG_STEEL_ACOUSTIC_GUITA                    25  //钢弦吉他
```

ZHUHAI JIELI TECHNOLOGY